

Dataproc / Managed Service for Apache Spark

Table of Contents

- 1. Introduction 3**
- 2. What Dataproc solves 3**
- 3. Where Dataproc fits in a data pipeline 3**
- 4. Dataproc deployment options 4**
- 5. First look at the Dataproc UI 4**
- 6. Cluster, Job, Workflow and Batch 6**
- 7. Spark concepts you must understand before using Dataproc 6**
- 8. How a PySpark batch actually runs 7**
- 9. Creating a serverless Spark batch from the UI 7**
- 10. Batch ID, region and runtime version 7**
- 11. Batches screen: monitor serverless workloads 8**
- 12. Opening a batch: Summary, Details and Logs 9**
- 13. Reading Dataproc logs 10**
- 14. Practical PySpark example — GCS Bronze to Silver 10**
- 15. Managed cluster basics 11**
- 16. Creating a managed cluster 11**
- 17. Autoscaling and capacity planning 12**
- 18. Job submission to a managed cluster 12**
- 19. Serverless batch vs managed cluster — decision scenario 12**
- 20. IAM and service accounts 13**
- 21. Data access, networking and region 13**
- 22. Performance tuning fundamentals 13**
- 23. Common Spark failure patterns 14**
- 24. Reliability and reruns 14**

25. Airflow / Cloud Composer integration 15

26. Monitoring and operational checklist 15

27. End-to-end production scenario 16

28. How to choose the right Dataproc approach 16

1. Introduction

Dataproc is the familiar name many teams use for Google's managed Apache Spark service. In the current Google Cloud product experience, the service is called Managed Service for Apache Spark (formerly known as Dataproc). It provides two main deployment choices: serverless Spark workloads and managed Spark clusters. Serverless batches run Spark workloads without requiring you to provision a cluster; managed clusters provide deeper infrastructure control.

Learning approach: Start with the Spark execution model, then understand the Dataproc UI, then learn how a batch is created and monitored, and finally study cluster operations, security, cost and troubleshooting.

2. What Dataproc solves

Running Spark yourself means deciding how compute is created, how workers are configured, how networking and identities are attached, how logs are collected and how the environment is maintained. Dataproc provides managed execution options so data engineers can concentrate on the Spark application and the data flow.

Scenario: A small CSV can be processed with ordinary Python. A multi-hundred-GB dataset with expensive joins and aggregations is a better candidate for distributed Spark processing.

3. Where Dataproc fits in a data pipeline

Source → Dataproc / Spark → Curated data → Analytics

GCS Bronze → PySpark → GCS Silver → BigQuery

Dataproc is usually the processing layer rather than the final storage or analytics layer. Airflow/Cloud Composer can orchestrate the workload, while GCS stores files and BigQuery serves analytical queries.

4. Dataproc deployment options

The most important Dataproc decision is whether to use serverless Spark or a managed cluster.

Rule of thumb: Start with serverless when you only need Spark execution and do not need cluster-level control. Start with a managed cluster when you need a persistent environment, custom cluster configuration or compatibility with an existing cluster-oriented workload.

5. First look at the Dataproc UI

The supplied screenshot shows the current Managed Service for Apache Spark landing page. The navigation separates clusters, cluster jobs and workflows from serverless batches and interactive sessions.

Figure 1 — Dataproc / Managed Service for Apache Spark Overview screen from the supplied project.

Beginner tip: The first decision on this screen is usually simple: do I need to control a cluster, or do I only need to submit Spark work?

6. Cluster, Job, Workflow and Batch

Cluster = workplace Job = work submitted to the workplace Workflow = planned sequence Batch = serverless Spark workload

Important distinction: A batch is not the same object as a cluster job. A batch is submitted to managed serverless compute, while a cluster job targets a cluster you provision.

7. Spark concepts you must understand before using Dataproc

Why this matters: You cannot troubleshoot Dataproc well if you only understand the Google Cloud UI. Spark concepts explain why a job is slow, why shuffle appears, and why executors may run out of memory.

8. How a PySpark batch actually runs

1. Submit the batch with the Python file, region, runtime and other configuration.
2. Google Cloud validates the request and the required permissions.
3. Managed compute and the Spark/Python runtime are prepared.
4. The Spark driver starts and creates the execution plan.
5. The application reads input data such as GCS files.
6. Spark executes transformations across partitions.
7. Actions such as writes trigger the actual computation.
8. Output is written to the destination.
9. The batch ends in a final state such as Succeeded or Failed.

User / Airflow → Dataproc Batch → Driver → Executors → GCS / BigQuery

Scenario: A daily pipeline submits a PySpark file at 02:00. The code reads JSON from a Bronze GCS path, filters invalid records, standardizes timestamps, calculates order amount and writes Parquet to a Silver path.

9. Creating a serverless Spark batch from the UI

The current console supports creating a batch directly from Managed Service for Apache Spark. A beginner should think about the inputs before touching the form: what code will run, where it will run, what data it reads, what it writes and which identity it uses.

1. Open Google Cloud Console and select the correct project.
2. Open Managed Service for Apache Spark → Batches.
3. Click Create.
4. Choose the batch type, such as PySpark.
5. Select the region.

6. Provide a meaningful Batch ID.

7. Select the runtime version required by the application.

8. Provide the main Python file URI, typically a GCS path.

9. Add Python package/archive/JAR/file dependencies when required.

10. Provide application arguments.

11. Configure Spark/runtime properties only when the workload requires them.

12. Configure the service account and networking options when required.

13. Review the configuration and submit the batch.

Practical rule: Do not add every possible Spark property on the first run. Start with the minimum configuration needed for a successful, repeatable workload; tune only after observing the job.

10. Batch ID, region and runtime version

These three fields are easy to overlook but important for operations.

Scenario: If a batch reads from resources in asia-south1 but is accidentally submitted elsewhere, you may create unnecessary network movement or encounter a configuration/networking mismatch.

11. Batches screen: monitor serverless workloads

The supplied Batches screenshot is the main operational list for serverless Spark runs.

Figure 2 — Batches page showing successful and failed PySpark runs.

Performance scenario: If a job normally takes 3 minutes and starts taking 15 minutes, compare the input volume, runtime version, shuffle-heavy operations and resource behavior before simply increasing resources.

12. Opening a batch: Summary, Details and Logs

Selecting a batch opens its run-level details. Use this page differently depending on whether the batch succeeded or failed.

Figure 3 — Batch Summary/details page for a successful PySpark run.

Troubleshooting order: Status → View Logs → first meaningful error → permissions/input/output/configuration → external Spark details if needed.

13. Reading Dataproc logs

A beginner mistake is treating every log line as an error. Managed Spark startup logs contain many informational messages about images, Python environments, dependencies and Spark startup.

Good habit: Find the first meaningful failure. Later errors can be consequences of the original problem.

14. Practical PySpark example — GCS Bronze to Silver

The supplied KT scenario uses a daily e-commerce cleaning job. Raw JSON files land in a Bronze bucket and the PySpark application writes cleaned Parquet to a Silver location.

```
from pyspark.sql import functions as F
df = spark.read.json(input_path)
clean_df = (df.filter(F.col("order_id").isNull())
            .withColumn("amount", F.col("amount").cast("double"))
            .withColumn("order_date", F.to_date("order_timestamp")))
clean_df.write.mode("append").parquet(output_path)
```

Spark transformations are evaluated lazily; the write action causes the plan to execute. For large data, the important design questions are partitioning, shuffle behavior, join strategy, output file layout and resource requirements.

Scenario: If the input doubles but the job remains within its expected SLA, no change may be needed. If runtime suddenly increases because a join causes a huge shuffle, investigate the execution plan before scaling the cluster blindly.

15. Managed cluster basics

A managed cluster is appropriate when you need cluster-level control or a reusable Spark environment. Unlike a serverless batch, you explicitly create the cluster and choose its configuration.

When to use a cluster: Choose it when you need persistent/custom infrastructure, repeated jobs on the same environment, legacy Spark applications or deeper control over configuration.

16. Creating a managed cluster

The exact console fields vary by product version and configuration, but the decisions follow a stable pattern.

- 1. Open Managed Service for Apache Spark → Clusters.**
- 2. Click Create.**
- 3. Choose the region/zone strategy supported by the product.**
- 4. Choose the Spark/runtime version.**
- 5. Choose the machine type and disk configuration for the primary/master node.**
- 6. Choose worker node machine types and counts.**
- 7. Configure networking/subnet and required access.**
- 8. Choose the service account and required scopes/permissions.**
- 9. Configure optional components and custom settings only when needed.**
- 10. Add autoscaling where variable load justifies it.**
- 11. Review cost and lifecycle expectations.**
- 12. Create the cluster.**

Design warning: A cluster that is left running when no work is happening can become a cost problem. A persistent cluster needs a lifecycle strategy.

17. Autoscaling and capacity planning

Autoscaling policies let a managed cluster change worker capacity based on workload demand. The goal is to avoid paying for maximum capacity all the time while still meeting workload requirements.

Important: Autoscaling does not repair a badly designed Spark job. If a job is slow because of a skewed join or huge shuffle, adding workers can increase cost without fixing the root cause.

18. Job submission to a managed cluster

Cluster-based workloads are submitted as jobs to an existing cluster.

- 1. Upload the Spark/PySpark application and dependencies to a reachable location such as GCS.**
- 2. Identify the target cluster and region.**
- 3. Submit the job from the UI, CLI, API or an orchestration system.**
- 4. Provide the main class or main Python file and arguments.**
- 5. Monitor the job state and logs.**
- 6. Inspect Spark UI/history when deeper execution analysis is required.**

GCS script → Dataproc cluster → Spark driver → Spark executors → output

Example CLI: For cluster-based PySpark, Google documents commands such as `gcloud dataproc jobs submit pyspark` with the script path, cluster name and region.

19. Serverless batch vs managed cluster — decision scenario

Use the business requirement, not familiarity, to choose the mode.

20. IAM and service accounts

A Spark workload needs an identity to access Google Cloud resources. The person submitting a workload and the workload itself can have different permissions.

Troubleshooting example: If a batch reports permission denied reading GCS, identify the workload service account, the exact bucket/object and the missing permission before adding a broad role.

21. Data access, networking and region

A Spark job can be technically correct and still fail because the surrounding cloud architecture is incorrect. Region, subnet, firewall/network access, service identity and data location can all matter.

Scenario: The code works in a development project but fails in production because the production subnet has different egress restrictions. The root cause is network configuration, not PySpark logic.

22. Performance tuning fundamentals

Performance tuning should start from the Spark execution model rather than random property changes.

- 1. Measure the baseline runtime and input volume.**
- 2. Check whether a transformation introduces a large shuffle.**

- Common anti-pattern: Increasing cluster size without checking the Spark execution plan. More compute can hide a problem temporarily while increasing cost.

24. Reliability and reruns

Scenario: Airflow retries a Dataproc task after a transient failure. If the Spark application blindly appends output every time, the final dataset may contain duplicates. Idempotency is an application design responsibility, not just a scheduler setting.

Cloud Composer can orchestrate Dataproc workloads as part of a larger pipeline. Airflow schedules and tracks the workflow, while Dataproc performs the distributed Spark processing.

Why use this separation? It keeps orchestration, distributed processing and analytical storage as separate responsibilities. A DAG should not become the place where large data processing is performed directly.

26. Monitoring and operational checklist

A production e-commerce pipeline can combine Cloud Storage, Dataproc, BigQuery and Cloud Composer.

1. Raw order files arrive in a GCS Bronze location.

2. Airflow starts the processing workflow.

3. Dataproc runs the PySpark cleaning application.

- 4. The application removes invalid records and standardizes the schema.**
- 5. Clean data is written to a deterministic Silver path.**
- 6. A downstream BigQuery task validates or loads the analytical result.**
- 7. The workflow reports success only after the required outputs are validated.**

Failure scenario: If the Spark job fails because a package is missing, fix the dependency configuration rather than increasing compute. If it fails because a GCS permission is missing, fix IAM. If it becomes slow because of a shuffle-heavy join, inspect Spark execution before changing infrastructure.

28. How to choose the right Dataproc approach

Decision rule: Choose based on workload lifecycle and control requirements. Do not create a cluster just because clusters are familiar.

Production-Ready Completion Guide

This section follows the same learning progression used in the Managed Airflow documentation: introduction → architecture → core concepts → UI/setup → configuration → security → monitoring → troubleshooting → performance/cost → reliability → production design → end-to-end scenario → common mistakes → operational checklist.

A. Standard learning path

29. Security and identity design

Dataproc workloads need access to input/output systems and should use explicit service identities, controlled networking and least privilege.

- Use a dedicated production service account where appropriate.
- Review GCS/BigQuery permissions separately.
- Use private networking and controlled egress when required.

30. Cost and cluster lifecycle

Serverless batches and managed clusters have different cost behavior; cluster lifecycle must be intentional.

- Use serverless for intermittent workloads when cluster control is unnecessary.
- Use autoscaling when justified.
- Delete/stop non-production clusters.
- Review machine type, worker count, disk and runtime before scaling.

31. Dependency and runtime management

Spark applications can fail because dependencies are incompatible rather than because compute is insufficient.

- Pin important dependencies.
- Test Python/JAR/package compatibility.
- Version application artifacts.
- Use Spark UI/history to distinguish application and infrastructure failures.

32. Production monitoring and alerting

Monitor both the Google Cloud workload and Spark execution model.

- Track state, duration and failure rate.
- Inspect driver/executor memory, shuffle and skew.
- Monitor cluster utilization and autoscaling.
- Alert on repeated failures or SLA deviations.

33. Reliability and rerun strategy

Retries are safe only when the application avoids unintended duplicate output.

- Use deterministic output or controlled overwrite/reconciliation.
- Make input intervals explicit.
- Validate output after recovery.
- Document whether retries are safe.

34. Common production mistakes

- Idle clusters left running.
- Scaling before investigating shuffle/skew/driver memory.
- Broad IAM.
- Incompatible dependencies.
- Unsafe append-style reruns.
- Ignoring environment network differences.

35. Operational checklist

- Correct region/runtime/environment.
- Correct input/output paths.
- Identity and network validated.
- Dependencies version controlled.
- Worker sizing justified.
- Logs and Spark metrics available.
- Rerun strategy documented.
- Cluster lifecycle/cost controls defined.
- Downstream validation completed.

Official references

<https://docs.cloud.google.com/dataproc/docs>

<https://docs.cloud.google.com/dataproc/docs/concepts/overview>

<https://docs.cloud.google.com/dataproc/docs/concepts/cluster-mode>

Final learning outcome

After completing this document, the learner should be able to explain Managed Service for Apache Spark, create or configure its main resources, operate it from the Google Cloud console, troubleshoot common failures, and describe production controls.

Reference Tables

Requirement	Dataproc capability	Example
Distributed processing	Runs Spark workloads across compute resources.	Process millions of order records.
Serverless execution	Submit a batch without first managing a cluster.	Daily PySpark cleanup job.
Managed clusters	Create clusters when you need control and a reusable Spark cluster.	Shared Spark cluster for a team.
Scaling	Serverless resources scale for the workload; cluster resources use autoscaling.	BigQuery use autoscaling for peak load.
Integration	Works with GCS, BigQuery and other Google Cloud services.	Read GCS → transform → write BigQuery.

Service	Primary responsibility
Cloud Storage	Object/file storage.
Dataproc / Managed Service for Apache Spark	Distributed Spark processing.
BigQuery	Analytical warehouse and SQL processing.
Cloud Composer / Airflow	Workflow orchestration.

Area	Serverless batch	Managed cluster
Infrastructure	No persistent cluster to provision for the workload.	You create and manage the cluster.
Best for	New pipelines, one-off jobs, spiky or variable demand.	Persistent environments, legacy Spark workloads and deeper customization.
Operations	Less infrastructure administration.	More configuration and administration.
Scaling	Managed compute scales for the workload.	Cluster can use autoscaling policies.
Cost model	Workload execution/resources consumed.	Cluster uptime plus underlying resources.
Typical example	Daily 3-minute PySpark cleaning batch.	Shared Spark cluster used by several teams.

UI item	What it means	Why it matters
Overview	Landing page for the service.	Starting point for choosing a processing model.
Clusters	Managed Spark clusters.	Use when cluster control is required.
Jobs	Jobs submitted to clusters.	Track cluster-based Spark work.
Workflows	Repeatable multi-step cluster workflows.	Useful for production sequences.
Autoscaling policies	Rules controlling cluster scaling.	Matches compute to changing demand.
Batches	Serverless Spark workloads.	Run Spark without provisioning a cluster.
Interactive Sessions	Temporary interactive Spark environments.	Useful for exploration/testing.
Session Templates	Reusable session configuration.	Reduces repeated setup.
Metastore	Metadata/catalog services.	Useful for Spark table metadata.
Federation	Metadata/catalog integration.	Supports selected external metadata patterns.
Component exchange	Supported Spark components/integrations.	Useful when extending the Spark environment.

Concept	Simple meaning	Example
Cluster	Managed group of compute resources used by Spark.	Master + worker nodes.
Job	One Spark workload submitted to a cluster.	Run clean_ecom.py.
Workflow	Defined sequence of jobs/tasks.	Extract → clean → aggregate → load.

Concept	Simple meaning	Example
Batch	Serverless Spark workload.	Submit clean_ecom.py without a persistent cluster.
Interactive Session	Temporary interactive Spark environment.	Explore a dataset in a notebook/session.

Term	Simple explanation	Example
Spark	Distributed processing engine.	Process hundreds of GB of orders.
PySpark	Python API for Spark.	clean_ecom.py uses DataFrames.
Driver	Main application process coordinating the Spark job.	Creates execution plan and coordinates tasks.
Executor	Process that performs work assigned by the driver.	Processes data partitions.
Partition	Chunk of data processed in parallel.	One large dataset split into many partitions.
Transformation	Builds a new DataFrame/RDD or execution plan.	filter(), select(), join().
Action	Triggers execution of the plan.	count(), write().
Shuffle	Redistributes data between executors.	Often caused by groupBy and some joins.

Setting	Why it matters	Good practice
Batch ID	Identifies a workload run.	Use a meaningful, searchable naming pattern.
Region	Determines where the workload executes.	Keep it aligned with data location, networking and downstream services.
Runtime version	Controls the Spark/Python runtime stack.	Pin/test versions intentionally rather than changing them casually.

Column	Meaning	How to use it
Batch ID	Identifier for the workload.	Find and open a specific run.
Version	Spark/runtime version.	Compare runs across runtime changes.
Location	Region where the batch runs.	Verify regional design.
Status	Current/final state.	First health check.
Tier	Service tier.	Useful for configuration/cost review.
Engine	Execution engine.	Useful when comparing execution options.
Creation time	When the batch was created.	Identify the correct run.
Elapsed time	Runtime duration.	Track performance changes.
Type	Workload type such as PySpark.	Confirm what was submitted.
Autotuning Summary	Available tuning information.	Review during performance analysis.

Action	Purpose
Clone	Create another batch using the current configuration as a starting point.
Cancel	Stop a running batch when cancellation is available.
Delete	Remove the batch resource.
View Logs	Read execution logs.
Refresh	Refresh the displayed state.
Spark History Server	Inspect Spark history when configured and available.
Spark UI	Inspect deeper Spark execution information when available.

Log signal	What to do
ERROR	Inspect the surrounding lines and stack trace.
Exception / Traceback	Find the first meaningful exception, not only the final wrapper error.
Permission denied	Check the identity and target resource.
File not found	Verify the exact GCS URI/path and date partition.
Module not found	Check package/dependency configuration.
OutOfMemoryError	Investigate data size, partitioning, joins and memory pressure.
Connection/network error	Check region, subnet/network access and service availability.

Stage	Example
Input	gs://ecom-bronze-landing-prod/orders/2026-08-27/*.json
Application	clean_ecom.py
Processing	Remove invalid rows, cast numeric columns, standardize timestamps and calculate order
Output	gs://ecom-silver-transformed-prod/orders/date=2026-08-27/
Orchestration	Cloud Composer / Airflow can submit the batch.
Monitoring	Batches → Status → Logs → Spark UI/history when available.

Cluster component	Role
Primary/master node	Hosts services that coordinate the cluster and Spark applications.
Worker nodes	Run distributed Spark tasks.
Secondary/worker capacity	Additional compute used for scalable workloads where configured.
Persistent disks	Provide local storage attached to nodes.
Network/subnet	Controls network placement and connectivity.

Situation	Better starting point
Stable workload, fixed demand	Manual capacity can be simpler.
Variable workload across the day	Consider autoscaling.
Occasional batch bursts	Consider serverless batches first.
Always-on shared cluster	Autoscaling can help match capacity to demand.

Scenario	Recommended starting point	Reason
Daily 3-minute PySpark batch	Serverless batch	No reason to keep a cluster running.
Team reuses a configured Spark environment every day	Managed cluster	Persistent shared environment may justify it.
Highly variable burst workload	Serverless batch	Compute is managed around the workload.
Legacy Spark workload depends on cluster-level managed	Managed cluster	More control/compatibility.
Need interactive development	Interactive session	Designed for exploration and testing.

Identity / control	What it affects
User/operator IAM	Whether a person can create/manage Dataproc resources.
Workload service account	Identity used by the Spark workload when accessing Google Cloud resources.

Identity / control	What it affects
GCS IAM	Whether the workload can read/write required objects.
BigQuery permissions	Whether the workload can read/write/query required datasets.
Network permissions/configuration	Whether the workload can reach required resources.

Check	Why
Region	Keeps compute aligned with data and downstream services.
Subnet/network	Determines connectivity to required services.
Private access	May be required by organizational network policy.
Service account	Controls resource access.
GCS path	Determines the actual input/output objects.

Problem	Likely area to investigate
Driver out of memory	collect(), large driver-side objects, bad aggregation behavior.
Executor out of memory	Partition size, skew, joins, executor memory requirements.
Long shuffle stage	groupBy, join, repartition and data skew.
Too many small output files	Partitioning and output strategy.
Input not found	GCS URI/path/date partition.
Module/package missing	Dependency packaging/runtime configuration.
Permission denied	Service account/IAM.
Slow job after data growth	Input volume, partitioning, skew, shuffle and resource size.

Risk	Safer approach
Appending the same partition twice	Write deterministically or replace/reconcile the target interval.
Duplicate output records	Use stable keys and deduplication where appropriate.
Partial output after failure	Use staging paths or atomic/deterministic write patterns.
Manual rerun after success	Know exactly which input interval and target are being processed.

Check	What to verify
Input	Expected GCS objects exist.
Batch/Job	Correct ID, region and runtime.
Identity	Correct service account and permissions.
Status	Succeeded / Running / Failed.
Logs	First meaningful error identified if failed.
Elapsed time	Within expected SLA.
Output	Correct location and expected record/file count.
Spark execution	No abnormal shuffle, skew or memory behavior.
Cost	No unnecessary persistent cluster usage.
Recovery	Rerun strategy understood.

Requirement	Start with
One-off/serverless Spark batch	Serverless Batch
Daily batch with highly variable load	Serverless Batch
Persistent shared Spark environment	Managed Cluster
Legacy cluster-oriented Spark application	Managed Cluster
Interactive exploration	Interactive Session
Repeatable multi-step cluster workflow	Workflow Templates / orchestration

Stage	Expected outcome
1. Introduction	Explain the service, problem and role.
2. Architecture	Explain components and flow.
3. Core concepts	Explain hierarchy, execution and terminology.
4. UI and setup	Create resources and recognize key settings.
5. Configuration	Configure runtime, identity, network and scaling.
6. Security	Apply IAM, encryption, secrets and least privilege.
7. Monitoring	Use logs, metrics and alerts.
8. Troubleshooting	Move from symptom to evidence to root cause.
9. Performance and cost	Relate design to throughput, latency and cost.
10. Reliability	Handle retries, duplicates and recovery.
11. Production design	Apply CI/CD, environment separation and ownership.
12. End-to-end scenario	Explain the service in a realistic pipeline.